

1Q) Given N array elements, check if ^{bool} there exists a pair (i, j) such that $\underbrace{arr[i]}_a + \underbrace{arr[j]}_b = k$ & $i \neq j$

arr:-

	0	1	2	3	4	5	6	7	8	9
arr	8	9	1	-2	4	5	11	-6	7	5

	i	j	arr[i]	arr[j]
--	---	---	--------	--------

K=11:	4	8	4	7
-------	---	---	---	---

K=6:	0	3	8	-2
------	---	---	---	----

K=22:	6	6	11	11
-------	---	---	----	----

Idea 1)

Check all pairs.

```
for(int i=0; i<N; i++) {
```

```
    a = arr[i], b = k - a
```

```
    for(int j=i+1; j<N; j++) {
```

```
        if(arr[j] == b) {
```

```
            return true
```

```
        }
```

```
    }
```

```
}
```

```
return false
```

TC - $O(N^2)$, SC - $O(1)$

① Create hashmap $\langle \text{int} \rangle \text{hs}$

② Insert entire data in hs

HS: { 8, 9, 1, -2, 4, 5, 11 }
 { -6, 7 }

K=11	a	b(k-a)	b is in hs
------	---	--------	------------

8	3	X
---	---	---

9	2	X
---	---	---

1	10	X
---	----	---

-2	13	X
----	----	---

4	7	✓
---	---	---

K=6	a	b(k-a)
-----	---	--------

8	-2	✓
---	----	---

Optimized by
hashset

arr:-

i	i	i	i	i	i	i			
0	1	2	3	4	5	6	7	8	9
8	9	1	-2	4	5	11	-6	7	5

K=22

a b = K - a

8 14 x

9 13 x

1 21 x

-2 24 x

4 18 x

5 17 x

11 == 11 ✓

2

freq of 11 == 1

is: { 8, 9, 1, -2, 4, 5, 11 }
{-6, 7}

Obs: If we know frequency,
we can solve the problem.

1) Hashmap < int, int > hm

2) iterate & insert in hm

hm: { 8:1 -2:1 11:1
9:1 4:1 -6:1
1:1 5:2 7:1

K=22

a b

8 14 x

9 13 x

1 21 x

-2 24 x

4 18 x

5 17 x

{ 11 == 11 } x

Edge case: a == b
if (freq(b) > 1)

a = b

freq(a) > 1

pair exist

Eg 7 5 2 5 9 5

HM: { 7:1 9:1
 5:3
 2:1 }

k=10

a	b	
7	3	x
(5 == 5)		

if(freq(s) > 1)

Pseudo Code:-

1) Create hashmap & insert arr[]

2) for(int i=0; i < N; i++) {

 int a = arr[i]

 int b = k - a

 if(a != b) {

 if(b is in hm) {

 return true

 }

 }

 else if(a == b) {

 if(hm[b] > 1) {

 return true

 }

 }

}

return false

TC:- O(N)

SC:- O(N)

Note:-

Search	} - constant O(1) for 1 operation
insert	
update	
delete	
size()	

Using Hashset :-

arr :-

	0	1	2	3	4	5	6	7	8	9
	8	9	1	-2	4	5	11	-6	7	5, 11

Idea :- When we are at i^{th} index, all elements from $[0, i-1]$ should be in the hashset

K = 22

a	b $\rightarrow K - a$	HS :- { }
8	14 X	insert 8
9	13 X	{8} insert 9.
1	21 X	{8, 9} insert 1
-2	24 X	{8, 9, 1} insert -2
4	18 X	{8, 9, 1, -2} insert 4
5	17 X	{8, 9, 1, -2, 4} insert 5
11	11 X	{8, 9, 1, -2, 4, 5} insert 11
		{8, 9, 1, -2, 4, 5, 11}

$\hookrightarrow$ 11 11 ✓

Pseudo Code

```
Hashset <int> hs
for(int i=0; i<N; i++) {
    int a = arr[i]
    int b = k - a
    if (b is in hs) {
        return true
    }
    hs.insert(a)
}
return false
```

TC: $-O(N)$

SC: $-O(N)$

TODO

Q1) Calculate No of i, j , such that $arr[i] + arr[j] = k$.

TODO

Q2) Check if there exists a pair i, j such that
 $arr[i] - arr[j] = k$ & $i \neq j$

Q3) Given N array elements, calculate No of distinct elements in every subarray of size k.

arr: -

	0	1	2	3	4	5	6	7	8	9
	2	4	3	8	3	9	4	9	4	10

k = 4 :-

$[0, 3] : 4$
 $[1, 4] : 3$
 $[2, 5] : 3$
 $[3, 6] : 4$
 $[4, 7] : 3$
 $[5, 8] : 2$
 $[6, 9] : 3$

Print no of distinct elements in all window of size k

$[0 \quad k-1]$ → Reverse Sliding Window session

$N - k - 0 + 1$
 $(N - k + 1)$
subarrays

$[N - k \quad N - 1]$

Idea:

For every window, get No of distinct elements using hashset

int i = 0, j = k - 1

while (j < N) {

hashset <int> hs

for (int k = i; k <= j; k++) {

hs.insert(arr[k])

}

print(hs.size())

i++

j++

}

TC: $(N - k + 1) * k : O(N^2)$

Put $k = N/2$

$(N - \frac{N}{2} + 1) (\frac{N}{2})$

$(\frac{N}{2} + 1) (\frac{N}{2})$

am: -

0	1	2	3	4	5	6	7	8	9
2	4	3	8	3	9	4	9	4	10

$k=4$

~~{2, 3, 8, 9}~~ ✓
 {3, 8, 9, 4}

		removed	added	HS	HS size
Initially	[0, 3]			{2, 4, 3, 8}	4
	[1, 4]	0	3	{ 2 , 4, 3, 8}	3
	[2, 5]	1	5	{ 4 , 3, 8, 9}	3
	[3, 6]	2	6	{ 3 , 8, 9, 4}	3
					(wrong)

Idea) If we remove an element, entirely all occurrences of same elements are also removed.

// Hashmaps with Sliding Window

arr :- 2 4 3 8 3 9 4 9 4 10

HashMap

$\begin{matrix} 0 & k-1 \\ \checkmark & \checkmark \end{matrix}$
[0 3]

remove

add

{2:1, 4:1, 3:1, 8:1} 4

$\begin{matrix} i=1 & j=k \end{matrix}$
[1 4]

0

4

delete from hm

{2:0, 4:1, 3:2, 8:1} 3

[2 5]

1

5

delete from hm

{4:0, 3:2, 8:1, 9:1} 3

[3 6]

2

6

delete from hm

{3:1, 8:1, 9:1, 4:1} 4

[4 7]

3

7

delete from hm

{3:1, 8:0, 9:2, 4:1} 3

[5 8]

4

8

{3:0, 9:2, 4:2} 2

[6 9]

5

9

{9:1, 4:2, 10:1} 3

Given N elements & K

HashMap <int, int> hm

```
for(int i=0; i < K; i++) {  
    // arr[i] insert in hashmap  
    if(arr[i] is in hm) {  
        // update val by 1  
        hm[arr[i]] ++  
    }  
    else { // insert arr[i] with val 1  
        hm.insert({arr[i], 1})  
    }  
}
```

print(hm.size())

int i=1, j=K

while(j < N) {

// subarray [i ... j]

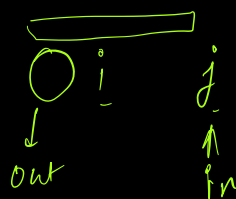
// i-1 is going out

hm[arr[i-1]] --

if(hm[arr[i-1]] == 0) {

hm.remove(arr[i-1])

}



}

```
// j is coming in
```

```
if (arr[j] is in hm) {
```

```
    hm[arr[j]]++
```

```
}
```

```
else {
```

```
    hm.insert(arr[j], 1)
```

```
}
```

```
print(hm.size())
```

```
i++
```

```
j++
```

```
}
```

TC: $O(N)$
SC: $O(N)$